

DM_Toolkit – Data Migration and Validation Toolkit

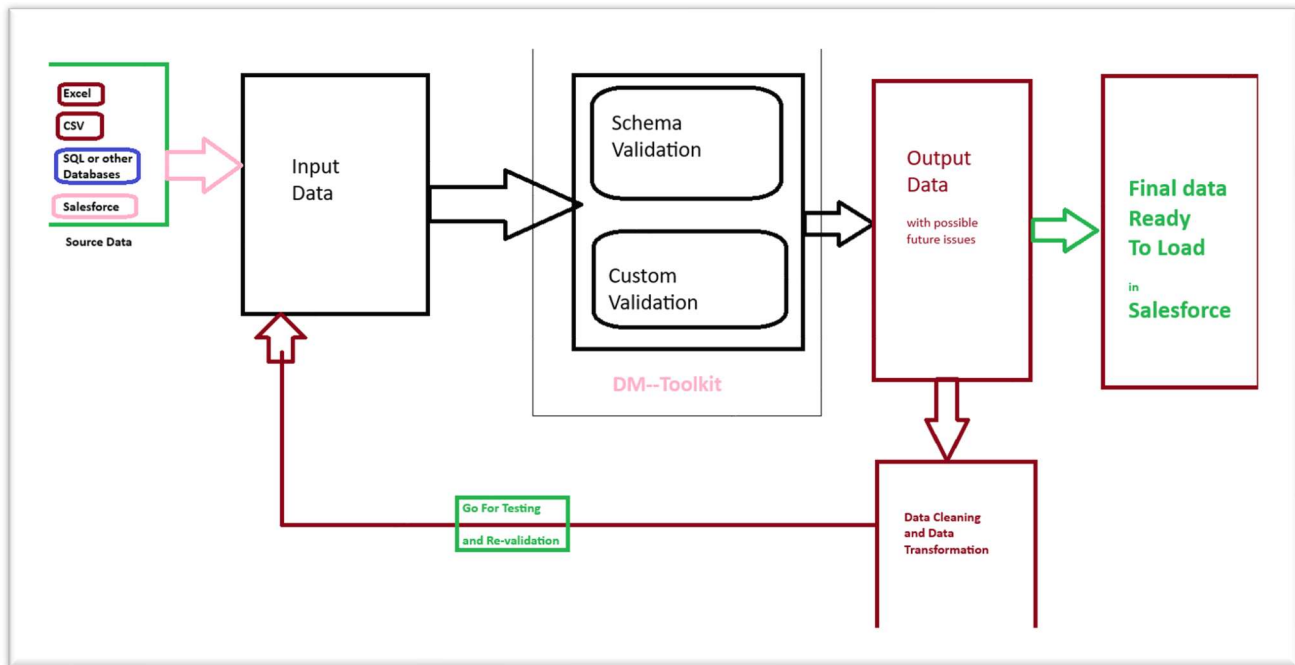
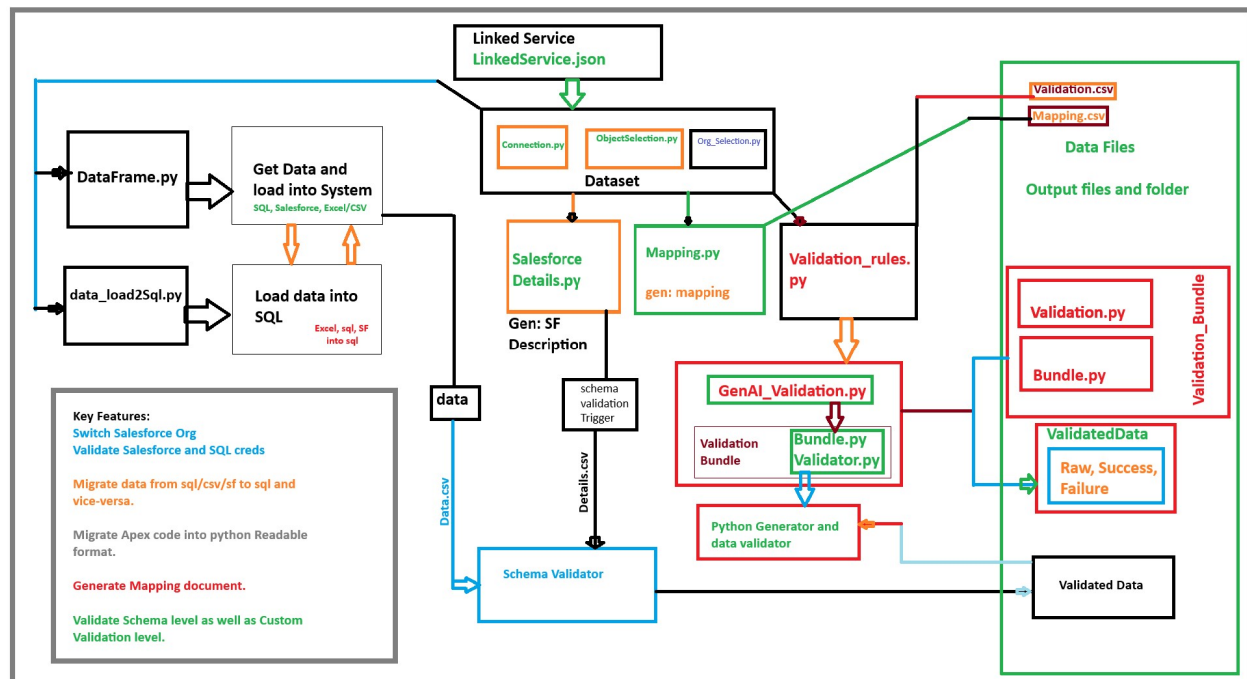


Table of Contents

1. Introduction
2. Architecture of DM_Toolkit
3. Requirements
4. DataSet and Connection Scripts
5. Schema-Level Validation
6. Custom Validation Using GenAI
7. Mapping Document
8. Other Tools
9. Pros and Use Cases
10. Future Tasks
11. Key Features
12. How to Run the Project

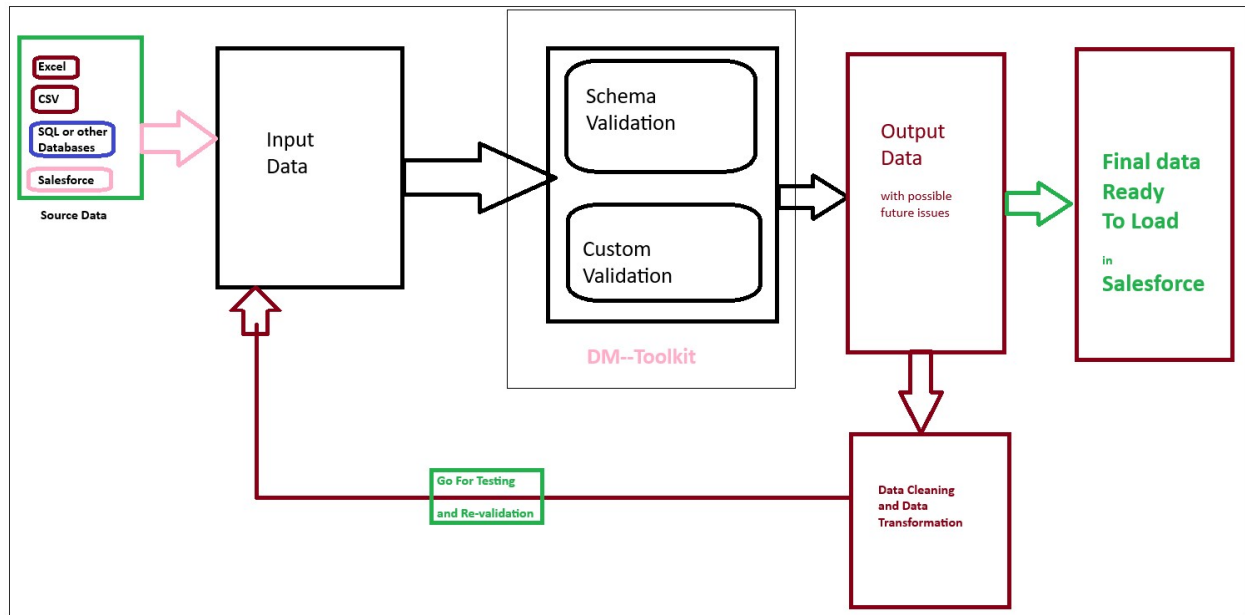
Introduction

The DM Toolkit project simplifies data migration by providing tools for data validation and transfer. It validates data against Salesforce Schema-Level Validation and Custom Validation rules for selected Salesforce objects. The toolkit supports fetching data from Salesforce, SQL, Excel, or CSV files and loading it into various systems as needed.



Architecture of DM_Toolkit

The DM Toolkit is a modular system that connects with Salesforce, SQL databases, and file-based sources (Excel/CSV). It uses scripts for connection management, object selection, data extraction, validation, and mapping. The toolkit leverages libraries for data manipulation, Salesforce interactions, and SQL connectivity, with GenAI-powered features to convert Salesforce Apex validation rules into Python-based logic.



Requirements:

The following Python libraries are required for the DM_Toolkit project:

Library	Purpose
json	Reading and manipulation of JSON files
sqlalchemy	SQL database connectivity and engine
simple_salesforce	Salesforce-related activities
pandas	Data manipulation and analysis
concurrent.futures	Parallel execution for performance

To install these libraries:

- Press Windows + R, type "cmd", and press Enter to open the Command Prompt.
- Navigate to the requirements.txt file location using the CD command.
- Run the command: `pip install -r requirements.txt`

DataSet and Connection Scripts

Connection.py This script connects to SQL databases and Salesforce, validating credentials and linked services. Use it to connect to your own Salesforce org or SQL database.

Object_selection.py This script lets you select the Salesforce object to work with for validation or data migration.

org_selection.py This script allows you to select the Salesforce org for data operations.

linkedservices.json This file stores connection details for services like Salesforce and SQL. Add your own services to this file for use in the project.

Dataframe.py This script fetches data from Salesforce, SQL, Excel, or CSV files and generates data files for validation or loading. It can extract data from one Salesforce system and save it as a CSV for migration to another system.

Validation Rule - Requirement:

To enforce data quality by preventing incorrect or incomplete data entry in Salesforce.

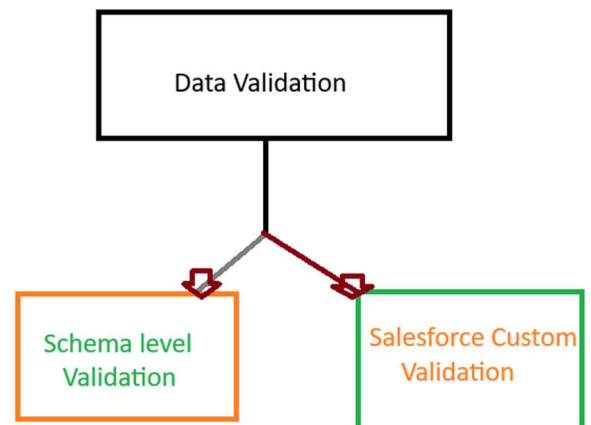
Validation rules in Salesforce ensure data entered meets specific criteria before saving.

Pros:

- Ensures data accuracy and consistency
- Prevents user errors at data entry time
- Enforces business rules automatically
- Reduces need for manual data cleanup

Types:

Generally, 2 types of Validation we are facing:



Schema-Level Validation

The toolkit performs schema-level validation for Salesforce objects with the following rules:

- Fields must not be Null.
- Unique columns must have no duplicate records.
- Email validation.
- Data format validation.
- Picklist validation.
- Data type validation.
- Required field validation.
- Reference field validation (ensuring referenced objects exist).

Salesforce_Details.py This script generates details for a selected Salesforce object, producing:

Salesforce_objectname.xlsx: Contains sheets for Field Names, Validation, and Triggers.

Details.csv: Provides schema-level details.

Validation.csv: Lists validation rules applied to the object.

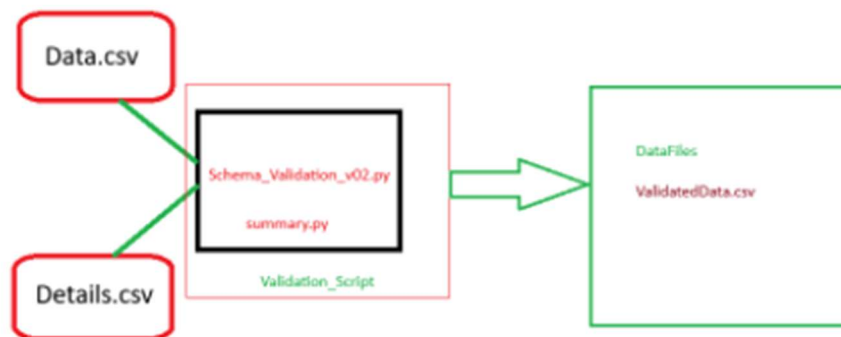
Working:

To get Schema level data validation, below are the steps and points to follow:

- Run **Salesforce_details.py** to get description of salesforce object like fields, validation and trigger w.r.t. object selection.
- Once you get **details.csv** Run **Schema_Validation_V02.py** to validate your data with respect to details.csv and object selection.

Requirement:

1. **Data.csv**:- If you have data in excel/csv/sql/salesforce then fine, else you must extract data and load into your system. We have already made scripts that will fetch data from different source system and load into our system (Datafiles folder) or sql depend upon requirement.
2. ***Details.csv**: - this file was generated by salesforce_details.py and its important for schema level validation.



Custom Validation Using GenAI

Custom validation for Salesforce objects includes:

Fetching validation rules and saving them as:

Validation.csv using Validation_Rules.py.

Converting Apex-based Error Condition Formulas to Python (Pandas) using GenAI_Validation.py.

Generating two Python files in the Validation Bundle folder:

Bundle.py: Converts Apex code into Python-readable format and creates validation functions.

Validator.py: Validates data using functions from Bundle.py and the selected input data file.

Creating two folders:

Validation Bundle: Contains Bundle.py and Validator.py.

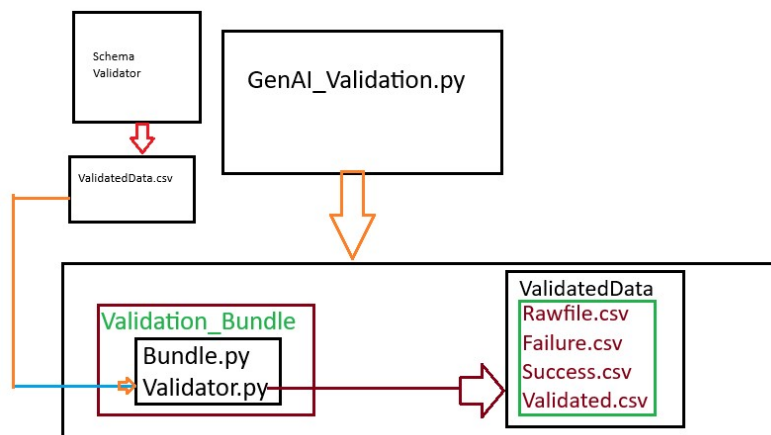
Validation Data: Stores validation rules fetched from Salesforce.

The ValidatedData folder contains:

Raw Data or Schema Validated Data file

Success Data file

Failure file (Schema-validated data with issues)



Mapping Document:

The Mapping.py script generates a JSON mapping document for the selected Salesforce object, required for data loading. This document can be manually updated and used to load data into Salesforce with dataload.py.

Other Tools:

data_load2sql.py This script loads data from CSV, Excel, or Salesforce query files into a SQL database

Pros and Use Cases

- Fetch data from SQL or CSV and create Pandas DataFrames.
- Extract Salesforce data and save as Excel/CSV or load into SQL.
- Migrate data between Salesforce systems by extracting and saving as CSV.
- Validate data against schema and custom rules for data integrity.

Future Tasks

- Support data extraction from Azure, MongoDB, and Google Drive (CSV/Excel).
- Enhance validation and mapping for additional data sources.

Key Features

- Select data source (Salesforce, SQL, Excel/CSV).
- Switch data points for flexible handling.
- Create mapping documents for specific objects based on user instructions.
- Validate data against schema and custom rules.

How to Run the Project

Run dataframe.py if you don't have data then fetch data from salesforce, sql.

*Run Salesforce_Details.py to capture object description in your system.

*Run Schema_Validation_v02.py to validate data and save schema level validation output of your raw data.

*Run validation_rules.py to get Error Condition formula for Object.

*Once you get Error Condition formula then Run **GenAI__Validation.py** to get prompt for entire custom validation in single python file (Python creates Python) Called Bundle.py and Validator.py to validate custom validation on your data.csv.

*Open Bundle.py to transform language.

*Run Validator.py to validate data against custom Validation of salesforce, it will generate 3 files, Raw File + Success (No transformation needed), Failure (Need to recheck data).

Additionally, if you wish to get data into sql and want to transform in sql only, you can run dataload2sql.py to get data in sql

Additionally, if you wish to get mapping of salesforce object, run mapping.py to get standard mapping, you can later update this based upon your data.